Enterprise RAG Architecture for the Tax Authority

Four Modules, Nine Questions

Module 1 · Ingestion & Knowledge

Q1 — Hierarchical chunking & verbatim citation
Q2 — Vector DB, HNSW, memory math

Module 2 · Retrieval Strategy

Q3 — Hybrid BM25 + dense, RRF fusion
Q4 — Cross-encoder reranking cascade

Module 3 · Agentic RAG & Self-Healing

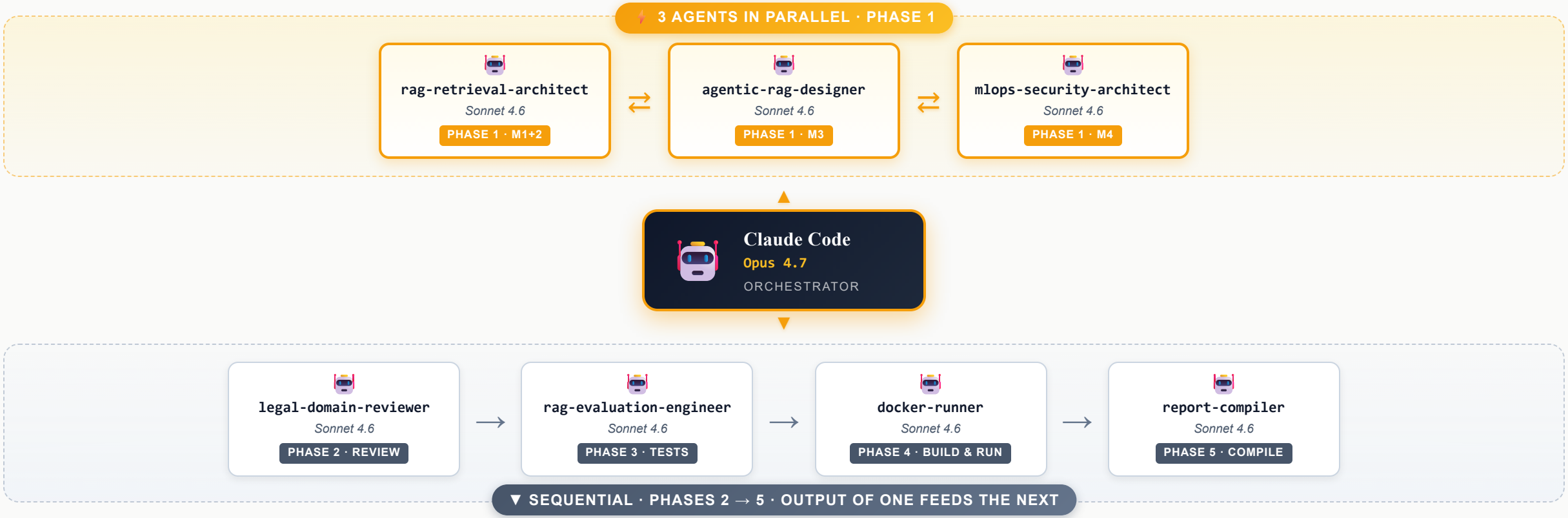
Q5 — Query transformation routing
Q6 — CRAG state machine, grader, fallback

Module 4 · Production Ops & Security

Q7 — Role-bound semantic cache
Q8 — Mathematically leak-proof RBAC
Q9 — CI gates & observability

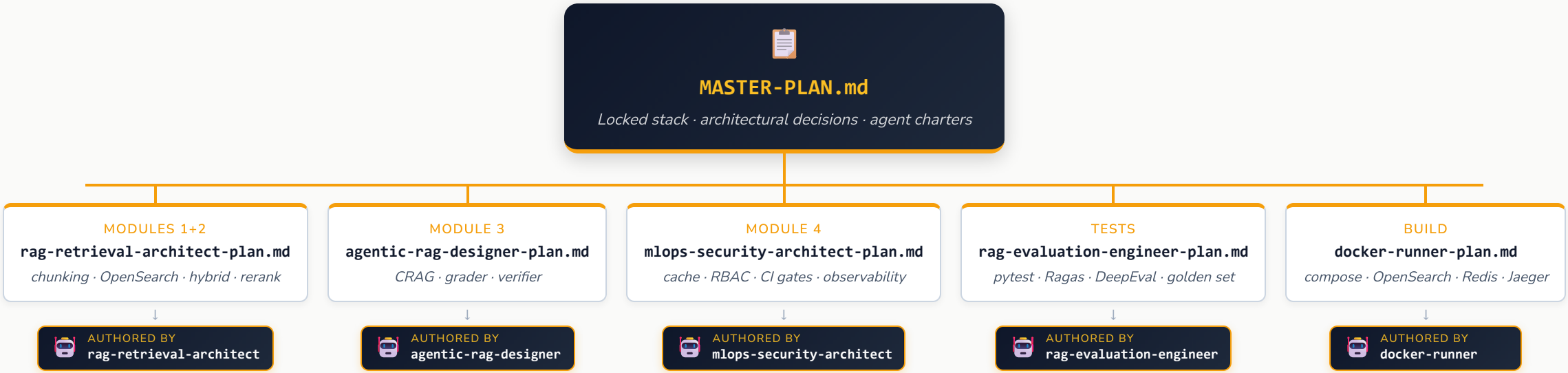
Multi-Agent System — Orchestrator + 7 Specialists

One orchestrator (Opus 4.7) delegates to **seven specialist agents** (Sonnet 4.6). Gold-bordered cards ran in parallel (Phase 1); the rest are sequential.



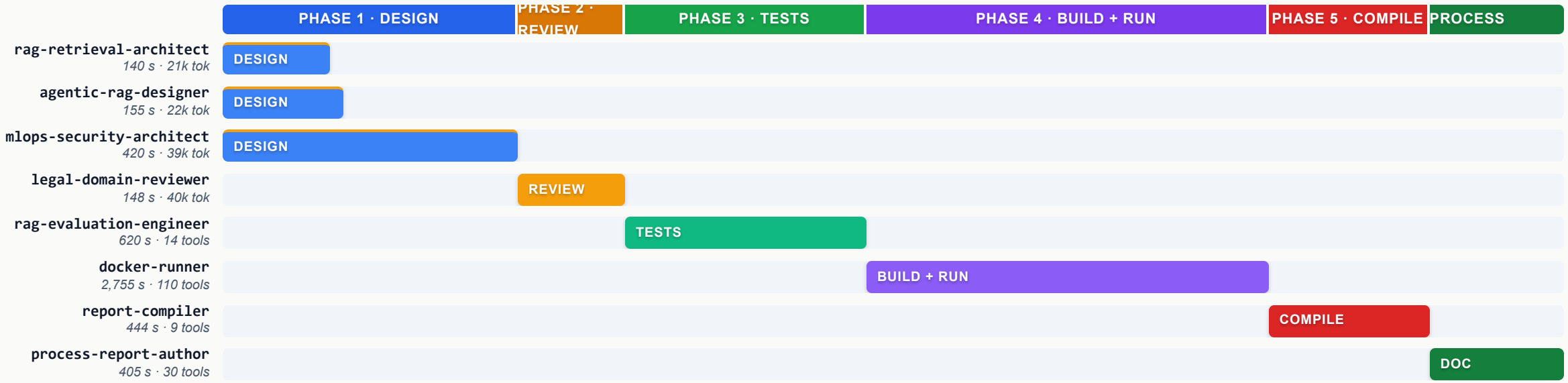
Plans — Master Plan + 5 Sub-Plans

Every agent works from a written plan in `plans/`. The master plan locks the stack and decisions; each sub-plan refines its module.



Agent Timeline — 7 Agents, 6 Phases, ~131 min Active

Phase 1 ran three designers in parallel (1.7× speed-up). Subsequent phases serialize because each consumes upstream output.



Parallelism factor: 1.7×

Combined design time 715 s · wall-clock 420 s (limited by mlops-security-architect)

Final: 173/173 tests green

docker-runner debugged 15 failures → 0 over 5 iterations · never weakened an assertion

1

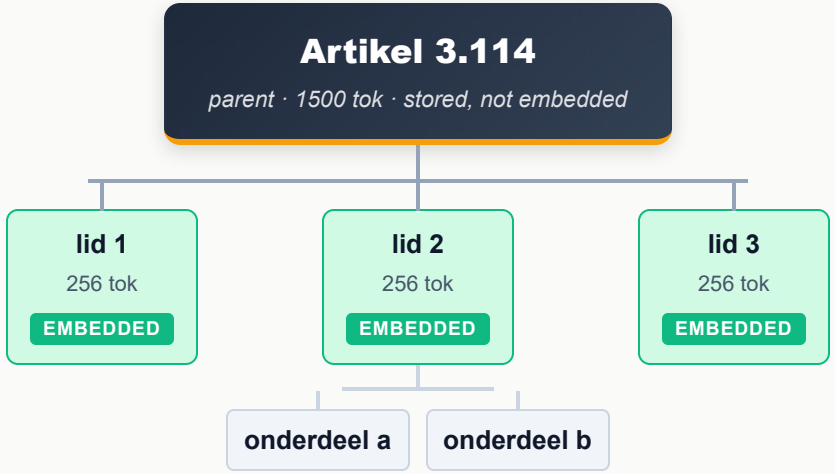
Module 1

Ingestion & Knowledge Structuring

How raw legal text becomes retrievable, citable evidence.

Q1 · Chunking — Structure-Aware Hierarchy

"How do you ensure the LLM knows a chunk belongs to 'Article 3.114, Paragraph 2'?"



① Structure-aware split

Separators are legal markers, not character counts: `^Artikel \d+` → `^\d+\.` (lid) → `^[a-z]\.` (sub-item) → `^\d+°` (sub)

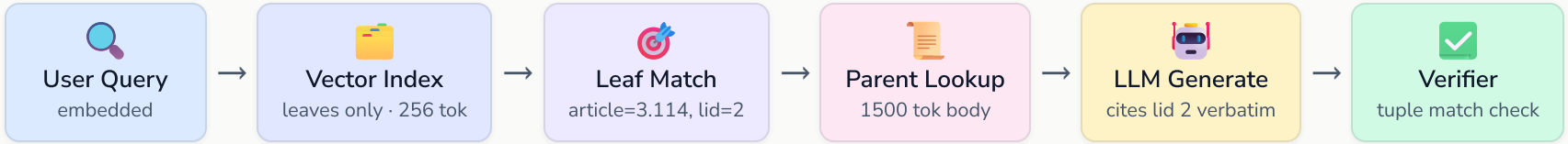
② Metadata as fields, not text

Each leaf carries `article`, `lid`, `sub-item` as filterable keywords plus `hierarchy_path: "ELI/wet/IB2001/artikel/3.114/lid/2"`

What makes it a chunk is that it's 256 tokens or less and it's the deepest node in that branch. The metadata fields just record *where in the hierarchy* that leaf lives:

```
{ "article": "3.114", "lid": "2", "sub_item": "b" }   ← sub-item level chunk
{ "article": "3.115", "lid": null,  "sub_item": null  }  ← article-level chunk (no deeper structure)
{ "article": "3.116", "lid": "1",  "sub_item": null  }  ← lid-level chunk
```

Q1 · Embed the Leaf, Serve the Parent



Zero-hallucination guarantee

The citation is *propagated* from a database field through the entire pipeline — never generated from LLM parameters. If the verifier can't match the emitted `(article, lid)` tuple against retrieved metadata, the answer is **rejected**.

KEY CONFIG

```
Leaf:     256 tok (tiktoken) · 32-tok overlap
Parent:   1500 tok (stored, not embedded)
Legis:    Article / lid / sub-item / sub
Caselaw:  facts / considerations / ruling
Embed:    Cohere multilingual-v3 · 1024-d
```

Q2 · Vector DB Choice — Security First

"Which Vector DB, what HNSW parameters, for 500K documents and 20M+ chunks?"

✓ OpenSearch (Lucene HNSW)

`efficient_filter` runs RBAC clause **inside** graph traversal. FIOD chunks never visited for helpdesk users → no empty-set inference, no timing leak.

✗ Pinecone Serverless

Filter applied **post-traversal**. Mathematically unsafe for zero-leak requirement.

✗ pgvector

Same problem — filter is post-traversal. Empty-set inference attack works.

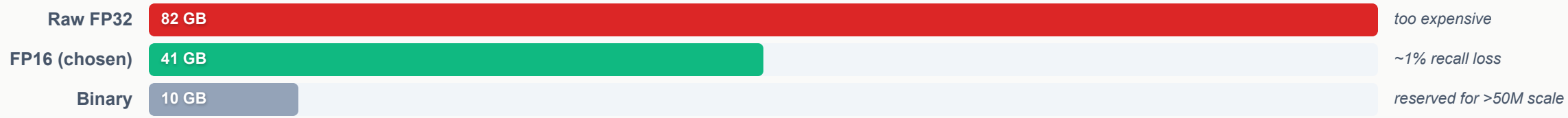
HNSW parameters

Parameter	Value	Reason
`m`	32	Recall ≥ 0.97 at 20M × 1024-d. m=16 → 0.92, m=48 → +50% RAM for marginal gain
`ef_construction`	256	Below 512 (super-linear build cost), above 128 (poor connectivity)
`ef_search`	128	Runtime-tunable to 64 under load via `_settings`, no reindex

PARAMETER	⬆️ HIGHER VALUE GIVES	⚠️ COST OF HIGHER VALUE
`m`	better connectivity · better recall	more memory · slower build
`ef_construction`	better index quality	slower indexing / build
`ef_search`	better search recall	slower query latency

Q2 · Memory Math — Why FP16

20M chunks × 1024-d vectors. The memory bill is the deciding factor.



Operational layout

12 primary shards · 1 replica · 3 dedicated master nodes
Fits 12 × 3.5 GB across `m6g.4xlarge` data nodes

Hot / warm tiering

Active years on SSD (FP16) · historical legislation on HDD (binary OK after 90 days zero-query) · DLS shard-level backstop

End-to-end latency budget: p95 ≤ 1,100 ms
— leaves 400 ms margin against 1,500 ms TTFT cap

2

Module 2

Retrieval Strategy

Two retrievers, one fusion, one reranker — each chosen for what the next can’t do.

Q3 · Hybrid Search — Both Arms Required

"How do you combine BM25 and dense retrieval? Which fusion strategy, and why?"

✗ BM25 alone fails on

`aftrekbaarheid werkruimte`

Vocabulary mismatch — no token overlap with corpus. Semantic meaning lost.

+

✗ Dense alone fails on

`ECLI:NL:HR:2023:123`

Embedding distance is weak for arbitrary identifiers. A legally distinct ruling can outscore the exact one.

=

✓ Hybrid fixes both

RRF fuses rank lists — no score normalisation needed

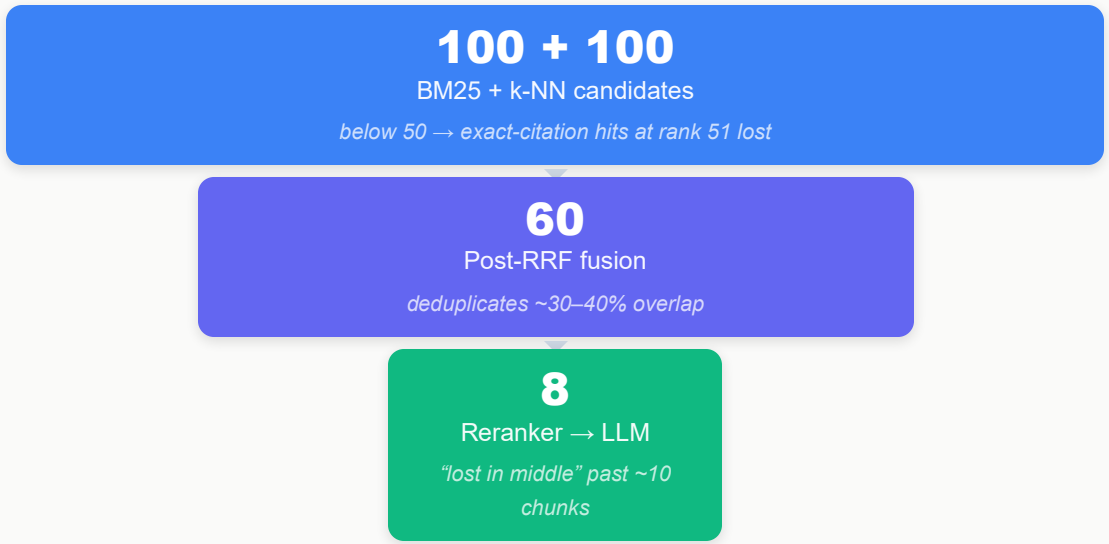
Fusion · RRF `k=60` — not alpha-weighted BM25 scores are unbounded; cosine similarity is bounded — you cannot blend them directly. RRF is purely rank-based and degrades gracefully when one retriever returns nothing.

ADAPTIVE REGEX ROUTER — MICROSECONDS, ZERO LLM CALLS

QUERY PATTERN	BM25 WEIGHT	DENSE WEIGHT	REASON
`ECLI:NL:[A-Z]+:\d{4}:\d+ or Artikel\s+\d+`	0.7	0.3	Exact citations must not be drowned by semantic noise
`Everything else (conceptual queries)`	0.5	0.5	Balanced — semantic and keyword equally valuable

Q4 · Reranking Cascade — Top-K Funnel

"Which reranking strategy and model? What are the Top-K parameters?"



Why two stages
Bi-encoders scale to 20M vectors but can't reason about query–document interaction. **Cross-encoders** take both as one input — query attends to every document token. Much more precise, but you can't run them over 20M. You **can** run them over 60.

Model: Cohere `rerank-v3-5` via Bedrock
✓ Verified multilingual: Dutch tax doc → 0.88, off-topic English → 0.02
✓ Runs on Bedrock — FIOD content never leaves AWS boundary
✓ Single API call returns sorted top-K

Fallback — what happens when reranking is too slow

The Bedrock rerank call is wrapped in `asyncio.wait_for` with a 300 ms hard timeout. If it times out, the pipeline skips reranking and returns the top-8 results directly from RRF fusion — so the user still gets an answer, just with slightly lower precision. The event is logged as `rerank_skipped=true` and visible in the Jaeger trace.

3

Module 3

Agentic RAG & Self-Healing

A graph, not a chain — with explicit loop guards and structural refusal paths.

Q5 · Query Transformation — One Transform, Picked Deterministically

"How do you handle complex multi-part tax questions? Decomposition or HyDE?"

Strict rule: **exactly one transform per query**, picked deterministically from raw text via regex. **No chaining.**

1 **Direct** — NO TRANSFORM
Exact identifier in query: ECLI:, art., lid, sub-item
Transforming would dilute BM25 precision

2 **Decompose** — MULTI-QUERY
Conjunction of >= 2 legal concepts: zowel... als, en, alsmede
Each sub-question gets its own retrieval and grading

3 **HyDE** — CLOSES VOCAB GAP
Vague / concept-only, no identifiers
Haiku writes hypothetical statute paragraph in corpus vocab -> embed -> retrieve

4 **Step-back** — RECOVERY
Re-entry after grader returns Irrelevant
Abstract to a higher principle, then re-narrow

Why no chaining:
destroys latency predictability and amplifies errors. LLM hallucinates in step 1 → HyDE amplifies it in step 2 → wrong corner of embedding space retrieved with high confidence.

Q5 · Worked Example — Decompose

"Kan een freelance vertaler in 2022 zowel thuiswerkkosten als reiskostenvergoeding aftrekken?"

`zowel…` `als` triggers **Decompose** →

Q1 — Home office costs

Wet IB 2001 art. 3.16 / 3.17

Grader:
Relevant

Q2 — Travel reimbursement

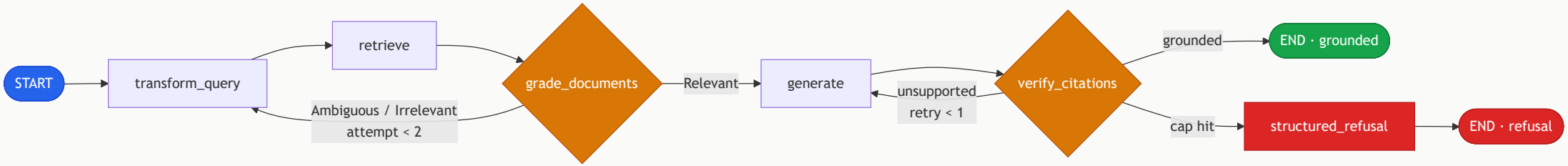
Jurisprudentie woon-werkverkeer ZZP

Grader:
Irrelevant

The generator answers Q1 and
explicitly discloses
that Q2 could not be grounded.
Never invents Q2.
That disclosure *is* the zero-hallucination guarantee.

Q6 · CRAG State Machine — Topology

"Design a CRAG state machine in LangGraph. Grader. Fallback policy."



Loop guards

`attempt_count ≤ 2`
`gen_retry_count ≤ 1`

State

Pydantic model with explicit counters — graph, not chain

Worst case — 6 Haiku calls, then fails closed

- 1× HyDE transform — writes hypothetical statute paragraph
- 3× Grader — attempt 0 → 1 → 2 (cap)
- 2× Verifier NLI — one per generate attempt

Q6 · The Grader

Configuration

Model: Haiku 4.5 via Bedrock
temperature = 0
DETERMINISTIC

Tool-use forced
guarantees JSON schema output, no freetext fallback
STRICT

Calibrated
against 50 golden triples at launch + after every corpus amendment batch
VERIFIED

System prompt rule:
"Grade only on chunks provided. No external knowledge. `missing_aspects` lists every uncovered part of the question."

Output schema

```
{
  "verdict": "Relevant | Ambiguous | Irrelevant",
  "confidence": 0.0–1.0,
  "reason": "string",
  "missing_aspects": ["string", ...]
}
```

Key rule:
confidence below
0.6
on Relevant verdict
auto-downgrades to Ambiguous
.

Forcing tool use eliminates an entire class of malformed-output bugs. The 0.6 floor is calibrated against the golden set.

Q6 · Exact Fallback Policy

Verdict	`attempt_count`	Action
Relevant	any	Proceed → `generate` → `verify_citations` → END
Ambiguous	0 or 1	Step-back rewrite → re-retrieve → re-grade
Ambiguous	2 (cap)	`generate_with_disclosure` — answer what's grounded, flag missing aspects
Irrelevant	0 or 1	Decompose/rewrite → re-retrieve **with elearning scope added** → re-grade
Irrelevant	2 (cap)	`structured_refusal` — return closest hits + missing aspects as JSON. **Never call generator.**

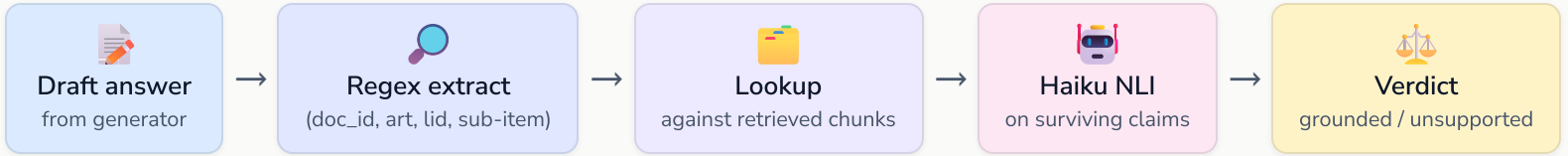
Why disclosure on Ambiguous-cap:

Fiscal inspectors usually want partial answers with gaps flagged, not outright refusals.

Why refusal on Irrelevant-cap:

Generating when retrieval truly failed twice is the textbook hallucination scenario. `closest_hits` still pass through RBAC redaction guard.

Q6 · Citation Verifier — The Second Guarantee



If any claim is unsupported:

Generation **retries once** with stricter prompt

Then **refuses** if still unsupported

The structural rule

The verifier never returns `final_answer` when `grounded=False`.

Backbone of the `Citation Accuracy = 1.00` CI gate.

4

Module 4

Production Ops, Security & Evaluation

The non-negotiables: zero leaks, zero ungrounded citations, p95 ≤ 1.5 s.

Q7 · Semantic Cache — The Confused Deputy Trap

"Design a semantic cache. What cosine similarity threshold is safe?"

Stack: Redis Stack 7.4 (`VECTOR` HNSW field) · AWS ElastiCache Serverless · sub-millisecond KNN

❌ DON'T key on **`SHA256(query_embedding)`** alone

Confused-deputy leak: an FIOD analyst's cached answer would be served to a helpdesk user asking the same question. The cache sits **upstream**

of retrieval and generation — no downstream RBAC can stop it.

✓ Mandatory role-bound key

```
key = "rag:cache:" + SHA256(emb_bucket ‖ user_role ‖ classification_ceiling ‖ tax_year)
```

Different roles produce different digests for the same query.

Namespaces are disjoint by construction.

Q7 · Threshold — Floor 0.97, Default 0.98

The reasoning is **empirical**, not theoretical.

"Box 1 income tax rate for 2024" vs "Box 1 income tax rate for 2023"
embeds at cosine **≈ 0.955** against Cohere `embed-multilingual-v3`.

The year token is only ~5% of the 1024-dim signal. At threshold **0.95** they collide → cache returns wrong year. At **0.97** they miss → fresh retrieval returns correct 2024 answer. The **0.98 default** adds headroom for embedding-version drift.

Q8 · RBAC — The Math of Where to Filter

"At what stage of the pipeline must the filter occur to prevent leaks mathematically?"

The filter must occur
inside the HNSW graph traversal itself
.
Not after retrieval. Not even just before retrieval. Anywhere else mathematically leaks.

Why post-filter fails — two leak vectors

① Empty-set inference

If the natural neighbourhood is dominated by FIOD chunks, post-filter returns an empty set. The user learns classified material exists around their topic — an
existence disclosure
.

② Timing side-channel

HNSW greedily follows nearest neighbours. FIOD-dense neighbourhood → walk wanders deeper. Latency differential
15–80 ms
on 20M-chunk index — over many queries, reveals classified-material density.

The fix:

OpenSearch `efficient_filter` compiles the classification clause into a Lucene BitSet. HNSW checks each candidate in O(1) and
prunes forbidden nodes from the graph walk
. Formally, the user searches the subgraph $G' = \{d \in G : d.\text{classification} \in \text{allowed}\}$. FIOD chunks are
invisible
.

Q8 · Defense in Depth + Role Matrix

Three layers

1. **efficient_filter in HNSW + DLS shard-level** — PRIMARY
   Stops all graph leaks; DLS backstops query bugs
2. **App redaction guard** — SECONDARY
   ORD[chunk.cls] <= ORD[user.ceiling] before LLM context

Role matrix

Role	Filter	IAM Principal
Helpdesk	`[public]`	`rag-helpdesk-app`
Inspector	`[public, internal]`	`rag-inspector-app`
Legal	`[public, internal]`	`rag-legal-app`
FIOD	`[public, internal, fiod]`	`rag-fiod-app`

CI gate: RBAC Leak Rate = 0.00

Hard fail · checked
before
all other metrics

Q9 · CI Gates — Hand-Curated 500-Pair Golden Set

"How do you automatically evaluate before deploying a new embedding model or LLM?"

GitHub Actions CI · 500 pairs: legislation current/historical · case law · FAQs · ambiguous · RBAC red-team · adversarial

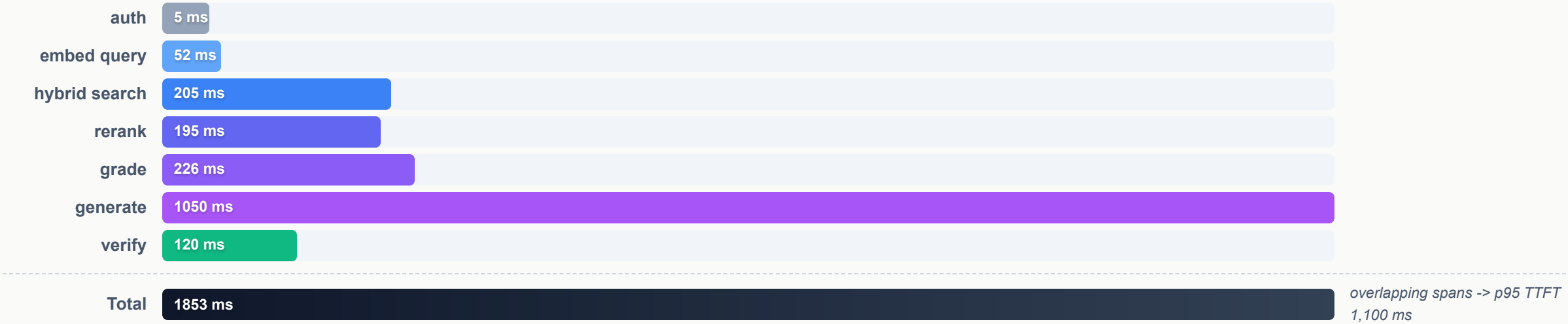
Metric	Framework	Threshold	Hard Fail?
Faithfulness	Ragas	≥ 0.95	✓
Context Precision	Ragas	≥ 0.85	✓
Context Recall	Ragas	≥ 0.90	✓
Answer Relevancy	Ragas	≥ 0.90	✓
Citation Accuracy	DeepEval (custom)	**= 1.00**	✓
Latency p95 TTFT	DeepEval	≤ 1,500 ms	✓
RBAC Leak Rate	DeepEval (red-team)	**= 0.00**	✓ — **first check, blocks before others report**

Faithfulness ≥ 0.95
— Ragas decomposes answer into atomic statements; each judged "inferable from context — yes/no". Below 0.95 the LLM is hallucinating at unacceptable rate.

Context Precision ≥ 0.85
— each retrieved chunk judged "relevant — yes/no", rank-weighted. A drop signals Module 1/2 problem (chunking, weights, rerank),
not
generation.

Q9 · Run-time Observability — Latency Waterfall

OpenTelemetry → Jaeger. Every request a trace; every node a span with mandatory attributes: `trace_id` · `user_role` · `classification_ceiling` · `attempt_count` · `latencies` · `grader_verdict` · `grounded` · `token_count`



Drift alerts

Signal	Threshold	Severity
`grader_verdict_irrelevant_rate` rising	> 15 pp over 7-day baseline	SNS — embedding drift
`rbac_guard_drop_count`	> 5 in 5 min	PagerDuty P2
`rbac_leak_count`	> 0 in any window	PagerDuty P1

Two structural guarantees, end-to-end

RBAC LEAK RATE

= 0.00

enforced inside HNSW · backed by DLS · audited via WORM S3

CITATION ACCURACY

= 1.00

propagated from DB metadata · verified post-generation · refusal on unsupported claims

Everything else — latency, recall, faithfulness — has margin.
These two have none.

Built with Claude Code & Slidev

Claude Code

Opus 4.7 orchestrator · Sonnet 4.6 sub-agents

The architecture, the test suite, the Docker stack, and this presentation were all designed and written by a multi-agent pipeline running inside Claude Code's CLI.

claude.com/claude-code

Slidev

Markdown · Vue · UnoCSS

Slides authored as a single `slides.md` file with custom Vue components for every diagram (AgentGraph, Funnel, Waterfall, LayerStack, PipeFlow, TreeChunk, AgentGantt). Hot-reload during development; PDF export for distribution.

sli.dev

Thank you.